

Fastag Fraud Detection

Evaluation Metrics & Analysis Report

Mentorless ML Internship Program

1. Executive Summary

This report documents the evaluation methodology, results, and key findings from the Fastag Fraud Detection classification project. Six classifiers were trained and compared using a stratified train/test split, class-imbalance-aware training, and 5-fold cross-validation. The best-performing model, a Decision Tree classifier, achieved a cross-validated F1-score of 0.992 on the fraud class.

The most significant finding of this project is not a modeling technique but a data characteristic: the `Fraud_indicator` label in this dataset is generated by an exact rule based on the mismatch between `Transaction_Amount` and `Amount_paid`, with zero exceptions across all 5,000 records. This is discussed in detail in Section 5, and directly shaped the final feature set used for modeling.

2. Dataset Overview

- Records: 5,000 toll transactions
- Class balance: 983 Fraud (19.7%) vs. 4,017 Not Fraud (80.3%) — a moderate imbalance
- Raw features: `Vehicle_Type`, `TollBoothID`, `Lane_Type`, `Vehicle_Dimensions`, `Transaction_Amount`, `Amount_paid`, `Vehicle_Speed`, `Geographical_Location`, `Vehicle_Plate_Number`, `Timestamp`
- Identifier columns (`Transaction_ID`, `FastagID`) and free-text location were dropped as non-generalizing

3. Feature Engineering

The following engineered features were derived from the raw columns:

- `state_code` — first two characters of `Vehicle_Plate_Number` (proxy for vehicle's registration state)
- `Hour`, `Day_of_Week`, `Month` — extracted from `Timestamp`

An earlier version of this pipeline also engineered `Payment_Diff` and `Payment_Ratio` (billed amount vs. amount paid). These were removed after the leakage analysis in Section 5 showed they directly encoded the target label rather than representing a learnable pattern.

4. Methodology

4.1 Handling Class Imbalance

Given the 80/20 class split, plain accuracy is a misleading metric — a model predicting "Not Fraud" for every transaction would score ~80% while catching zero fraud. All evaluation in this report focuses on precision, recall, and F1 for the Fraud class specifically.

- Stratified train/test split (80/20) to preserve the fraud ratio in both sets
- `class_weight='balanced'` for Logistic Regression, Decision Tree, Random Forest, and SVM
- `sample_weight` (via `compute_sample_weight`) for Gradient Boosting, which has no native `class_weight` parameter

4.2 Avoiding Data Leakage

- Categorical encoders (`LabelEncoder`) were fit on the training split only, then applied to the test split — categories unseen in training map to a reserved "unknown" code rather than leaking test-set information into the encoding

- 5-fold stratified cross-validation was run on the training data to confirm results were not the product of one favorable split

5. Key Finding: Data Leakage in Engineered Payment Features

During model comparison, Decision Tree, Random Forest, and Gradient Boosting all achieved a perfect 1.00 F1-score on the fraud class when a Payment_Diff feature (Transaction_Amount – Amount_paid) was included. A perfect score in fraud detection is not a good sign — it usually indicates a feature is leaking the label rather than the model finding a genuine pattern.

A direct check of the dataset confirmed this:

- Every one of the 983 rows labeled "Fraud" has Transaction_Amount $\neq$ Amount_paid
- Every one of the 4,017 rows labeled "Not Fraud" has Transaction_Amount = Amount_paid
- This holds with zero exceptions — the label is generated by this exact rule in the source dataset

Because of this, Payment_Diff and Payment_Ratio were removed from the final feature set. Transaction_Amount and Amount_paid remain as raw features, since they are genuinely present in the data, but the model must now use them (and other features) as inputs rather than being handed the label-generation formula directly. This is reflected in the more realistic — though still very strong — metrics reported in Section 6.

6. Model Comparison & Evaluation Metrics

All models below were trained on the same stratified 80/20 split, then cross-validated with 5 folds on the training portion. Sorted by cross-validated fraud-F1 (most reliable estimate of generalization):

Model	Accuracy	Precision (Fraud)	Recall (Fraud)	F1 (Fraud)	ROC-AUC (Fraud)	5-fold CV F1 (Fraud)
Decision Tree	0.998	1.00	0.990	0.995	0.995	0.992 (± 0.008)
Gradient Boosting	0.991	1.00	0.954	0.977	0.999	0.979 (± 0.014)
Logistic Regression	0.988	1.00	0.939	0.969	0.967	0.980 (± 0.010)
KNN	0.990	1.00	0.949	0.974	0.985	0.975 (± 0.007)
Random Forest	0.986	1.00	0.929	0.963	0.998	0.975 (± 0.009)
SVM	0.980	1.00	0.898	0.947	0.964	0.965 (± 0.012)

Confusion matrices for the top two models (test set, n=1000):

Decision Tree (selected model)

	Pred: Fraud	Pred: Not Fraud
Actual: Fraud	195	2
Actual: Not Fraud	0	803

Gradient Boosting

	Pred: Fraud	Pred: Not Fraud
Actual: Fraud	188	9
Actual: Not Fraud	0	803

Note: precision on the Fraud class is 1.00 for every model — none of the models ever raised a false fraud alarm on the test set. Differences between models come entirely from recall — how many actual fraud cases each model catches.

7. Explanatory Analysis: Feature Importance

Feature importances from the selected Decision Tree model confirm that, even without the explicit Payment_Diff feature, the model relies almost entirely on the raw amount fields — consistent with the leakage finding in Section 5:

Feature	Importance Score	Share of Total
Amount_paid	0.465	46.5%
Transaction_Amount	0.455	45.5%
TollBoothID	0.070	7.0%
Hour	0.0035	0.35%
Vehicle_Speed	0.0020	0.20%

Amount_paid and Transaction_Amount together account for roughly 92% of the model's decision-making. TollBoothID contributes a further 7%, suggesting some toll booths see disproportionately more fraud (worth a follow-up breakdown by booth if the underlying business wants to act on it). Vehicle type, lane type, vehicle dimensions, day-of-week, and month contribute negligibly — in this dataset, fraud is overwhelmingly a payment-amount phenomenon rather than a behavioral or demographic one.

8. Model Selection & Recommendation

The Decision Tree classifier achieved the highest cross-validated fraud-F1 (0.992) and is the model saved for deployment. Two caveats are worth noting for whoever maintains this model next:

- Single decision trees are more prone to variance across different random seeds/data samples than ensemble methods. Re-running this comparison with a different random_state showed Decision Tree and Gradient Boosting trading the top spot by a small margin.
- Gradient Boosting (F1 0.977, ROC-AUC 0.999, CV mean 0.979 ± 0.014) is a defensible alternative choice for production use if model stability across future data refreshes is prioritized over the last percentage point of fraud recall.

Recommendation: keep Decision Tree as the current deployed model given its measured performance, but re-validate model choice periodically as new transaction data arrives, since the small margin between the top models means the ranking is not guaranteed to be stable over time.

9. Deployment

The selected model is deployed via a Streamlit web application (app.py). The app loads the trained model, label encoders, target encoder, and exact training feature-column order directly from disk artifacts produced by the training script, so the deployed inputs always match what the model was trained on. Users input transaction details through a sidebar form and receive a real-time Fraud / Not Fraud prediction with the model's confidence score.

10. Conclusion

This project delivered a working fraud-classification pipeline evaluated with metrics appropriate to an imbalanced classification problem, and identified — rather than accidentally reported — a data leakage issue that would otherwise have produced a misleadingly perfect model. The final Decision Tree model catches 99.0% of fraudulent transactions on held-out data with no false positives observed, and is deployed for real-time scoring via Streamlit.